



5



10



1200 sp

عملي

كلية الهندسة المعلوماتية

السنة الثالثة

تمارين عن List

عملي مشترك



RB Informatics; 17/05/2026

محتوى مجاني غير مخصص للبيع التجاري

مبادئ الذكاء الصناعي

السلام عليكم في هذه المحاضرة لن نتعلم أفكار جديدة وانما ستكون مراجعة لفكرة List

1. التمرين الأول:

❖ اكتب predicate يقوم باستخراج العنصر الثاني من القائمة

الحل: الهدف هو استخراج العنصر الثاني فقط من قائمة، وتجاهل أي شيء آخر.

```
main.pl
File Edit Browse Compile Prolog Pce Help
main.pl
second_element([_, X | _], X).
Buffer saved in file 'main.pl' Line: 1
```

شرح الكود:

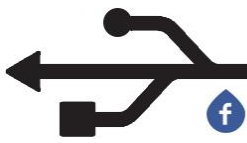
- **العنصر الأول:** نحن نعلم أن القائمة يجب أن تبدأ بعنصر ما، لكن قيمته لا تهمنا أبداً، لذلك لا داعي لتسميته أي نستخدمه كمتغير مجهول (`Anonymous Variable` (`_`))
- **العنصر الثاني:** `X` هذا هو العنصر المستهدف الذي نبحث عنه، قمنا بتسميته `X` لنحتفظ بقيمته.
- ووضعنا `|` التي تفصل بين العناصر الأولى المحددة وبين باقي القائمة (`Tail`) وبما أن باقي القائمة (سواء كان يحتوي على عنصر واحد، أو ألف عنصر، أو فارغاً) لا يهمنا، وضعنا بعد البار (`_`) لتجاهله تماماً.
- وعند الاستدعاء:

```
SWI-Prolog (AMD64, Multi-threaded, version 8.4.2)
File Edit Settings Run Debug Help
?-
% c:/Users/Dell/Desktop/telegram downloads/prolog/main.pl
compiled 0.00 sec, 1 clauses
?-
| second_element([a, b, c], X).
X = b.
?-
```

الوسيط الثاني للقاعدة: `X` قمنا بوضع نفس المتغير `X` أي إذا نجحت المطابقة، نجعل النتيجة المرجعة هي قيمة العنصر الثاني `X`

عند الاستدعاء `second_element([a, b, c], X)`:

- العنصر الأول `_` يطابق `a` يتم تجاهله



- العنصر الثاني X يطابق b
- باقي القائمة $_{}$ يطابق $[c]$ يتم تجاهله
- النتيجة المرجعة هي $X = b$

ملاحظة: بدون البار يفهم البرولوج أن القائمة يجب أن تتكون من ثلاثة عناصر فقط لا غير.

أمثلة:

- $[a, b, c]$ ستنتج (لأنها 3 عناصر) ويعطينا $X = b$
 - $[a, b]$ ستفشل وتعطي false لأنها أقل من 3 عناصر
- إذا قمنا باستدعاء القاعدة ومررنا قائمة فارغة [] النتيجة ستكون false

القاعدة تشترط لوجودها وجود عنصر أول وعنصر ثاني على الأقل، وبما أن القائمة الفارغة لا تحتوي على أي عناصر، تفشل عملية المطابقة (Pattern Matching) مباشرة

2. التمرين الثاني:

❖ اكتب predicate يسمى swap مهمته التأكد من تطابق قائمتين مع تبديل أول عنصرين فقط:

الحل:

```
main.pl
File Edit Browse Compile Prolog Pce Help
main.pl
swap([H, X | T], [X, H | T]).
Buffer saved in file 'main.pl' Line: 2
```

وعند الاستدعاء:

```
SWI-Prolog (AMD64, Multi-threaded, version 8.4.2)
File Edit Settings Run Debug Help
?- swap([1, 2, 3, 4], Result).
Result = [2, 1, 3, 4].
?-
```

نحن نريد الوصول إلى أول عنصرين تحديداً لتعديلهما، لذلك قمنا بتفكيك القائمة الأولى في الوسيط الأول لل predicate إلى ثلاثة أجزاء باستخدام البار |:

- H : يمثل العنصر الأول تماماً في القائمة الأصلية.
 - X العنصر الثاني: يمثل العنصر الذي يليه مباشرة.
 - T : يمثل باقي عناصر القائمة (كل شيء يأتي بعد العنصر الثاني)، وسميناه T لأننا سنحتاجه كما هو.
- بمجرد كتابة هذا السطر $[H, X | T]$ فهو يفهم تلقائياً أن هذه القائمة يجب أن تحتوي على عنصرين على الأقل لتنتج المطابقة.

❧ كيفية الوصول إلى المخرجات (القائمة الثانية)؟

في الوسيط الثاني، قمنا بإعادة بناء القائمة بالترتيب الجديد الذي نريده، مستخدمين نفس المتغيرات التي اخذناها من القائمة الأولى:

وضعنا X في البداية ليصبح هو العنصر الأول.

وضعنا H بعده مباشرة ليصبح هو العنصر الثاني (وبذلك تمت عملية التبديل بنجاح).

وضعنا البار T | نلصق باقي القائمة في النهاية دون أي تعديل على ترتيب عناصره

ماذا لو كانت القائمة تحتوي على عنصرين فقط؟

لو استدعيتِ swap([a, b], Result). ستنتج العملية أيضاً!

- ستكون a H
- ستكون b X
- T ستكون قائمة فارغة []
- النتيجة المرجعة ستكون [b, a | []] والتي يكتبها البرولوج ببساطة كـ [b, a]

3. التمرين الثالث:

❖ اكتب predicate الهدف منه هو تكرار كل عنصر من عناصر القائمة الأولى مرتين متتاليتين في القائمة الثانية أي مبني على مفهوم العودية (Recursion)

الحل:

```
main.pl
File Edit Browse Compile Prolog Pce Help
main.pl
twice([], []).
twice([H | T1], [H, H | T2]) :- twice(T1, T2).
```

○ عند الاستدعاء:

```
SWI-Prolog (AMD64, Multi-threaded, version 8.4.2)
File Edit Settings Run Debug Help
?-
* c:/Users/Dell/Desktop/telegram downloads/prolog/main.pl
compiled 0.00 sec, 1 clauses
?- twice([a, b], L).
L = [a, a, b, b].
?-
```

❧ حالة التوقف (Base Case):

أبسط حالة ممكنة هي ان تكون القائمة الفارغة []

إذا كانت القائمة الأولى فارغة، فإن تكرار عناصرها مرتين سينتج عنه أيضاً قائمة فارغة []

↪ الحالة العودية (Recursive Case):

- في المدخلات: $[H | T1]$ قمنا بفصل العنصر الأول H عن باقي القائمة $T1$
- في المخرجات: $[H, H | T2]$ بما أن الهدف هو مضاعفة كل عنصر، أخذنا H وكررناه مرتين متتاليتين في بداية القائمة الجديدة. $[H, H | ...]$ أما باقي القائمة الثانية فقد أسميناه $T2$ وهو الذي سيحتوي على مخرجات تكرار باقي العناصر لاحقاً

شرط الاستدعاء الذاتي: $\text{twice}(T1, T2)$:- نخبر البرولوج : لكي تنجح هذه العملية، خذ ما تبقى من القائمة الأولى $T1$ وقم بمضاعفة عناصره لتبني لنا القائمة $T2$ عند الاستدعاء. $\text{twice}([a, b], L)$:

1. الخطوة الأولى:

- يطابق $[a, b]$ مع البنية $[H | T1]$ ، فيصبح $H = a$ و $T1 = [b]$
- شكل القائمة الثانية يصبح $[a, a | T2]$
- الآن يستدعي البرولوج نفسه لمعالجة $T2$ $\text{twice}([b], T2)$

2. الخطوة الثانية:

- يطابق $[b]$ مع البنية، فيصبح $H = b$ و $T1 = []$
- شكل القائمة $T2$ يصبح $[b, b | T3]$
- يستدعي البرولوج نفسه مجدداً $\text{twice}([], T3)$

3. الخطوة الثالثة (الوصول لحالة التوقف):

- يطابق $\text{twice}([], T3)$ مع القاعدة الأولى $\text{twice}([], [])$ فيعرف مباشرة أن $T3 = []$

4. مرحلة التجميع العكسي:

- يعوض $T3$ في الخطوة الثانية، فنجد أن $T2 = [b, b | []]$ أي $T2 = [b, b]$
- يعوض $T2$ في الخطوة الأولى، فنجد أن النتيجة النهائية: $L = [a, a, b, b]$

4. التمرين الرابع:

اكتب predicate يقوم بدمج قائمتين عن طريق أخذ عنصر من الأولى ثم عنصر من الثانية بالتناوب

```
main.pl [modified]
File Edit Browse Compile Prolog Pce Help
main.pl [modified]
combine([], [], []).
combine([H1 | T1], [H2 | T2], [H1, H2 | T3]) :- combine(T1, T2, T3).
combine([H | T], [], [H | T]).
combine([], [H | T], [H | T]).
```

```

SWI-Prolog (AMD64, Multi-threaded, version 8.4.2)
File Edit Settings Run Debug Help
?- combine([a, b], [1, 2], L).
L = [a, 1, b, 2]

```

٤ حالات التوقف - (Base Cases):

عند دمج قائمتين، قد لا تكونان بنفس الطول، أو قد تنتهيان معاً لذلك نحتاج قواعد تخبر البرولوج ماذا يفعل إذا فرغت إحدى القائمتين:

- السطر الأول. $\text{combine}([], [], [])$: إذا كانت القائمة الأولى فارغة والثانية فارغة، فإن ناتج دمجهم هو قائمة فارغة بالتأكيد.
- السطر الثالث. $\text{combine}([H | T], [], [H | T])$: إذا فرغت القائمة الثانية أولاً وبقيت عناصر في القائمة الأولى، فلا داعي للمتابعة بالتناوب فنأخذ القائمة الأولى كما هي $([H | T])$ ونضعها مباشرة كناتج نهائي.
- السطر الرابع. $\text{combine}([], [H | T], [H | T])$: فالعكس تماماً إذا فرغت القائمة الأولى أولاً وبقيت عناصر في القائمة الثانية، نأخذ القائمة الثانية كما هي $([H | T])$ ونضعها مباشرة كناتج نهائي.

٤ الحالة العودية الأساسية (السطر الثاني):

- المدخلات: نأخذ $H1$ من القائمة الأولى ونفصله عن $T1$ ونأخذ $H2$ من القائمة الثانية ونفصله عن $T2$
- المخرجات: $[H1, H2 | T3]$ نقوم بوضع $H1$ يليه مباشرة $H2$ في بداية القائمة الناتجة أما باقي القائمة فاسمها $T3$

الاستدعاء الذاتي: $\text{combine}(T1, T2, T3)$ - نطلب من البرولوج تكرار العملية على ما تبقى من القائمتين $T1$ و $T2$ لإنتاج باقي قائمة الحل $T3$

عند الاستدعاء $\text{combine}([a, b], [1, 2], L)$:

1. الخطوة الأولى (تطبيق الحالة العودية):

- القائمة الأولى $[a, b]$ تتفكك إلى $H1 = a$ و $T1 = [b]$
- القائمة الثانية $[1, 2]$ تتفكك إلى $H2 = 1$ و $T2 = [2]$
- قائمة النتيجة تبدأ بـ $[a, 1 | T3]$ يستدعي البرولوج $\text{combine}([b], [2], T3)$:

2. الخطوة الثانية (تطبيق الحالة العودية مجدداً):

- القائمة الأولى $[b]$ تتفكك إلى $H1 = b$ و $T1 = []$
- القائمة الثانية $[2]$ تتفكك إلى $H2 = 2$ و $T2 = []$
- قائمة النتيجة $T3$ تصبح $[b, 2 | T4]$



○ يستدعي البرولوج (combine([], [], T4):

3. الخطوة الثالثة (الوصول لحالة التوقف):

○ يستدعي (combine([], [], T4) فيطابق السطر الأول مباشرة، ويعرف أن $T4 = []$

4. التجميع العكسي:

○ يعوض $T4$ في الخطوة الثانية $T3 = [b, 2 | []]$ أي $[b, 2]$

○ يعوض $T3$ في الخطوة الأولى. $L = [a, 1 | [b, 2]]$

○ النتيجة النهائية المرجعة: $L = [a, 1, b, 2]$

5. التمرين الخامس:

❖ اكتب predicate يقوم بدمج القائمة الأولى كاملة ثم القائمة الثانية:

الحل:

```
main.pl
File Edit Browse Compile Prolog Pce Help
main.pl
conc([], L2, L2).
conc([H | T], L2, [H | R]) :- conc(T, L2, R).
Line: 1
```

• وعند الاستدعاء:

```
SWI-Prolog (AMD64, Multi-threaded, version 8.4.2)
File Edit Settings Run Debug Help
?-
% c:/users/dell/desktop/telegram downloads/prolog/main co
mpiled 0.00 sec, -2 clauses
?-
|   conc([a, b], [1, 2], L).
L = [a, b, 1, 2].
?-
```

⚡ حالة التوقف (Base Case):

أبسط حالة هي أن تكون القائمة الأولى فارغة تماماً $[]$

إذا كانت القائمة الأولى فارغة، وقمنا بدمجها مع أي قائمة ثانية $L2$ ، فإن النتيجة النهائية ستكون هي القائمة الثانية $L2$ نفسها دون أي تغيير

- **تفكيك القائمة الأولى:** $[H | T]$ نقوم بفصل العنصر الأول H عن باقي القائمة الأولى T
- **بناء قائمة النتيجة:** $[H | R]$ نحن نعلم أن H (العنصر الأول من القائمة الأولى) يجب أن يكون هو العنصر الأول في القائمة النهائية، لذلك وضعناه في الرأس مباشرة. $[H | ...]$ أما باقي قائمة النتيجة فقد أسميناه R اختصار لـ Result، وهو الجزء الذي لا نعرفه حالياً ومنتظر الحساب.
- **الاستدعاء الذاتي:** $\text{conc}(T, L2, R)$ - نخبر البرولوج من أجل حساب هذا الباقي R ، خذ ما تبقى من القائمة الأولى T ونضعه مع القائمة الثانية $L2$ في الاستدعاء القادم.

عند الاستدعاء. $\text{conc}([a, b], [1, 2], L)$:

1. الخطوة الأولى:

- القائمة الأولى $[a, b]$ تتفكك إلى $H = a$ و $T = [b]$
- قائمة النتيجة النهائية تكون قيمتها مؤقتاً $[a | R1]$
- يستدعي البرولوج $\text{conc}([b], [1, 2], R1)$

2. الخطوة الثانية:

- القائمة الأولى $[b]$ تتفكك إلى $H = b$ و $T = []$
- قائمة النتيجة للخطوة الحالية تصبح $R1 = [b | R2]$
- يستدعي البرولوج $\text{conc}([], [1, 2], R2)$

3. الخطوة الثالثة (حالة التوقف):

- الاستدعاء هو $\text{conc}([], [1, 2], R2)$. يطابق السطر الأول في الملف $(\text{conc}([], L2, L2))$ مباشرة.
- هنا يعرف البرولوج فوراً أن المتغير المقابل لـ $R2$ يجب أن يساوي القائمة الثانية، أي $R2 = [1, 2]$

4. مرحلة التجميع العكسي:

- يعوض قيمة $R2$ في الخطوة الثانية $R1 = [b | [1, 2]]$ وتساوي $[b, 1, 2]$
- يعوض قيمة $R1$ في الخطوة الأولى $L = [a | [b, 1, 2]]$
- النتيجة النهائية المرجعة: $L = [a, b, 1, 2]$

6. التمرين السادس:

❖ اكتب predicate يقوم بتحويل قائمة تحتوي على قوائم فرعية متداخلة إلى قائمة بسيطة (Flat):

الحل:

```
main.pl
File Edit Browse Compile Prolog Pce Help
main.pl
flatten([], []).
flatten(X, [X]) :- X \= [_|_], X \= [].
flatten([H | T], R) :- flatten(H, R1), flatten(T, R2), append
(R1, R2, R).
```

عند الاستدعاء:

```
SWI-Prolog (AMD64, Multi-threaded, version 8.4.2)
File Edit Settings Run Debug Help
% c:/users/dell/desktop/telegram downloads/prolog/main co
mpiled 0.02 sec, -1 clauses
?-
| flatten([a, [b, c], d], L).
L = [a, b, c, d].
?-
```

٤ حالة التوقف الأولى (السطر الأول):

أبسط قائمة متداخلة هي القائمة الفارغة [], والقيام ب Flat لها ينتج عنه قائمة فارغة أيضاً []

- حالة التوقف الثانية (التعامل مع عنصر مفرد (السطر الثاني)):

أثناء تفكيك القوائم، سنصل في النهاية إلى عناصر مفردة (مثل الحرف a أو الرقم 5 وليس قوائم) لذلك سنضعه داخل قائمة [X]

الشرط $X \setminus = []$: يضمن أن X ليس قائمة تحتوي على H أو T ، و $X \setminus = []$ يضمن أنه ليس قائمة فارغة هذا يعني أن هذا السطر يعمل فقط إذا كان X عنصراً عادياً (Atom أو Number)

٤ الحالة العودية:

$\text{flatten}(H, R1)$: نأخذ H ونضع نتيجته في قائمة اسمها R1 لو كانت H قائمة مثل [b, c] ستتحول إلى [b, c]، ولو كان عنصراً مثل a سيتحول إلى [a]

$\text{flatten}(T, R2)$: نأخذ T (باقي القائمة) ونضع نتيجته في قائمة مسطحة اسمها R2

$\text{append}(R1, R2, R)$: بعد أن أصبح لدينا R1 و R2 جاهزين نقوم بملصقهما معاً باستخدام append لإنتاج القائمة النهائية المفردة R

1. تتبع سريع - كيف يرى البرولوج المثال: $\text{flatten}([a, [b, c], d], L)$

2. تفكيك القائمة الكبيرة إلى $H = a$ و $T = [[b, c], d]$.

3. $\text{flatten}(a, R1)$: يطابق السطر الثاني مباشرة ويعطي $R1 = [a]$

4. $\text{flatten}([b, c], d], R2)$: سيعاد تفكيكه عودياً بنفس الطريقة لينتج في النهاية $R2 = [b, c, d]$

5. الدمج النهائي: يتم استدعاء $\text{append}([a], [b, c, d], L)$ لتركيب النتيجة النهائية: $L = [a, b, c, d]$

7. التمرين السابع:

❖ اكتب predicate يقوم بأخذ قائمة وعنصر معيناً، ويعيد القائمة بعد حذف كافة تكرارات هذا العنصر منها

الحل:

```
main.pl
File Edit Browse Compile Prolog Pce Help
main.pl
delete_all([], _, []).
delete_all([X | T], X, R) :- delete_all(T, X, R).
delete_all([H | T], X, [H | R]) :- H \= X, delete_all(T, X, R).
Make done Line: 4
```

○ وعند الاستدعاء:

```
SWI-Prolog (AMD64, Multi-threaded, version 8.4.2)
File Edit Settings Run Debug Help
?- delete_all([a, b, a, c], a, L).
L = [b, c]
```

٤ حالة التوقف (Base Case) السطر الأول:

إذا كانت القائمة الأصلية فارغة تماماً []، فمهما كان العنصر المراد حذفه (ولذلك وضعنا _ لأن قيمته لا تهمنا)، فإن النتيجة ستكون دائماً قائمة فارغة []

• الحالة العودية الأولى - عند مطابقة العنصر المراد حذفه (السطر الثاني):

هنا نقوم بتفكيك القائمة، فإذا وجدنا أن رأس القائمة يساوي تماماً العنصر المراد حذفه (استخدمنا نفس الحرف X في الرأس وفي الوسيط الثاني، وهذا يجبر البرولوج على المطابقة تلقائياً)

المخرجات: بما أننا نريد حذف X، فلن نضعها في قائمة النتيجة المخرجة. لذلك تركنا قائمة النتيجة بالطرف الأيسر مجرد متغير R ينتظر القيمة.

الاستدعاء الذاتي: أي تخطى هذا العنصر X الحالي، ويتابع البحث والحذف في T لتجد لنا القائمة النهائية R

٤ الحالة العودية الثانية - عند اختلاف العنصر (السطر الثالث):

إذا كان رأس القائمة عنصراً مختلفاً فنريد الاحتفاظ به لذلك قمنا بتسميته ب H والعنصر المراد حذفه X

الشرط: $H \neq X$: أي أن الرأس لا يساوي العنصر المراد حذفه

المخرجات: $[H | R]$: بما أن H عنصر سليم ولا نريد حذفه، يجب أن نحافظ عليه ونضعه مباشرة في رأس قائمة النتيجة الجديدة. $[H | ...]$

الاستدعاء الذاتي: نطلب من البرولوج الاحتفاظ ب H والانتقال لمعالجة وحذف التكرارات من بقية T لإنتاج الجزء المتبقي من الحل R

عند الاستدعاء. $delete_all([a, b, a, c], a, L)$:

1. الخطوة الأولى (تطبيق الحالة الثانية - السطر الثالث لأن a لا يطابق شيئاً مختلفاً، بل نريد مقارنتها:

○ هنا الرأس هو a والعنصر المستهدف a يطابق السطر الثاني. $delete_all([X|T], X, R)$

○ يتم إهمال a الأولى، ويتحول الاستدعاء إلى. $delete_all([b, a, c], a, L)$:

2. الخطوة الثانية (تطبيق الحالة الثالثة - السطر الرابع):

○ الرأس b لا يساوي المستهدف $a (b \neq a)$

○ يتم الاحتفاظ ب b في النتيجة $[b | R1]$: $L = [b | R1]$

○ يتحول الاستدعاء إلى الذيل $delete_all([a, c], a, R1)$:

3. الخطوة الثالثة (تطبيق الحالة الثانية مجدداً):

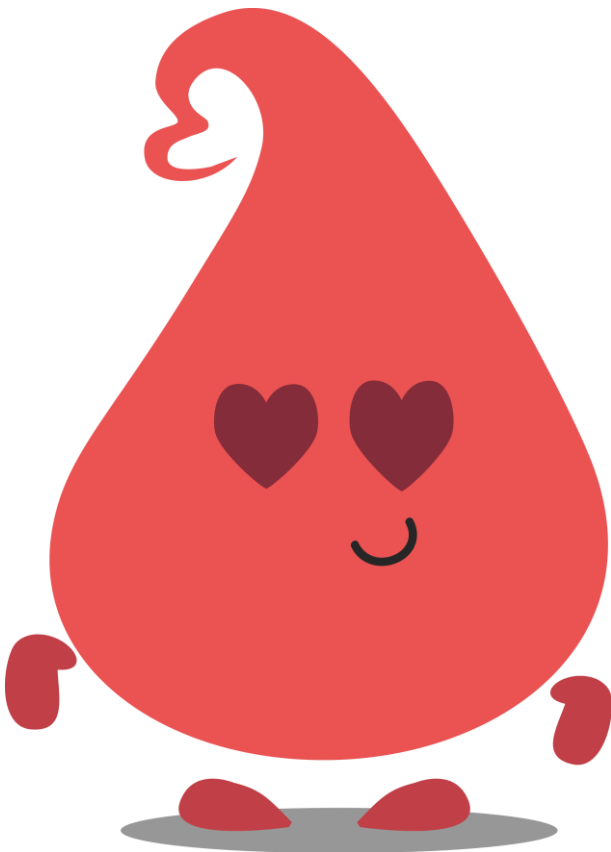
○ الرأس a يساوي العنصر المستهدف a

○ يتم إهمال a الثانية، ويصبح الاستدعاء $delete_all([c], a, R1)$:

4. الخطوة الرابعة (تطبيق الحالة الثالثة مجدداً):

○ الرأس c لا يساوي $a (c \neq a)$

- يتم الاحتفاظ بـ $c: R1 = [c \mid R2]$
- يتحول الاستدعاء إلى الذيل $:delete_all([], a, R2)$
- 5. الخطوة الخامسة (حالة التوقف):
 - القائمة فارغة تماثل السطر الأول $R2 = []$
- 6. التجميع العكسي للنتيجة:
 - نعوض $R2$ في الخطوة الرابعة $R1 = [c \mid []]$ أي $[c]$.
 - نعوض $R1$ في الخطوة الثانية $L = [b \mid [c]]$.
 - النتيجة النهائية المرجعة: $L = [b, c]$



وهنا تكون قد انتهت محاضرتنا
نلتقاكم في المحاضرات القادمة...

انتهت المحاضرة
<The end>